Kernel Resources



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Kernel Development News

Linux Weekly News

- https://lwn.net/
- The weekly digest off all Linux and free software information sources
- In depth technical discussions about the kernel
- Coverage of the features accepted in each merge window
- Subscribe to finance the editors ($7 / month)
- Articles available for non subscribers after 1 week.





Useful Online Resources

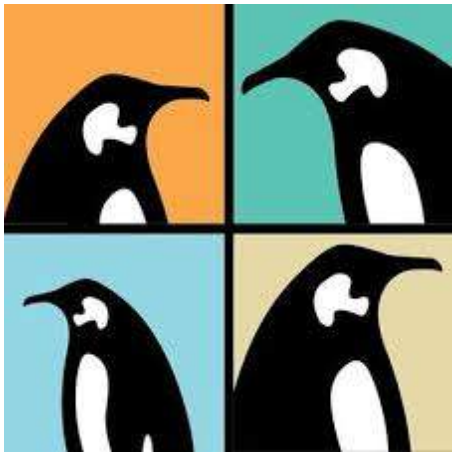
- Kernel documentation
  - https://kernel.org/doc/
- Linux kernel mailing list FAQ
  - https://subspace.kernel.org/etiquette.html
  - Complete Linux kernel FAQ
  - Read this before asking a question to the mailing list
- Linux kernel mailing lists
  - https://lore.kernel.org/
  - Easy browsing and referencing of all e-mail threads
  - Easy access to an mbox in order to answer to e-mails you were not Cc'ed to
- Kernel Newbies
  - https://kernelnewbies.org/
  - Articles, presentations, HOWTOs, recommended reading, useful tools for people getting familiar with Linux kernel or driver development.
  - Glossary: https://kernelnewbies.org/KernelGlossary
  - In depth coverage of the new features in each kernel release: https://kernelnewbies.org/LinuxChanges
- The https://elinux.org wiki



International Conferences (1)

- Embedded Linux Conference:
  - https://embeddedlinuxconference.com/
  - Organized by the Linux Foundation every year in North America and in Europe
  - Very interesting kernel and user space topics for embedded systems developers. Many kernel and embedded project maintainers are present.
  - Presentation slides and videos freely available on https://elinux.org/ELC_Presentations
- Linux Plumbers
  - https://lpc.events/
  - About the low-level plumbing of Linux: kernel, audio, power management, device management, multimedia, etc.
  - Not really a conventional conference with formal presentations, but rather a place where contributors on each topic meet, share their progress and make plans for work ahead.

Embedded Linux Conference





International Conferences (2)

- Kernel Recipes: https://kernel-recipes.org/
  - Well attended conference in Europe (Paris), only one track at a time, with a format that really allows for discussions.
- Embedded Recipes: https://embedded-recipes.org/
  - Same as Kernel Recipes, but focused on embedded topics (Nice).
- linux.conf.au: https://linux.org.au/linux-conf-au/
  - In Australia / New Zealand
  - Features a few presentations by key kernel hackers.
- Currently, most conferences are available on-line. They are much more affordable and often free.



LINUXCONFAU



After the course

Continue to learn:

- Run your labs again on your own hardware. The Nunchuk lab should be rather straightforward, but the serial lab will be quite different if you use a different processor.
- Learn by reading the kernel code by yourself, ask questions and propose improvements.
- Implement and share drivers for your own hardware, of course!

Hobbyists can make their first contributions by:

- Helping with tasks keeping the kernel code clean and up-to-date: https://kernelnewbies.org/KernelJanitors/Todo
- Proposing fixes for issues reported by the *Coccinelle* tool: `make coccicheck`
- Participating to improving drivers in drivers/staging/
- Investigating and do the triage of issues reported by Coverity Scan: https://scan.coverity.com/projects/linux



Contribute your changes

Recommended resources

- See process/submitting-patches for guidelines and https://kernelnewbies.org/UpstreamMerge for very helpful advice to have your changes merged upstream (by Rik van Riel).
- Watch the *Write and Submit your first Linux kernel Patch* talk by Greg. K.H: https://www.youtube.com/watch?v=LLBrBBImJt4
- How to Participate in the Linux Community (by Jonathan Corbet). A guide to the kernel development process

Last slides

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!





Last slide

Thank you!

And may the Source be with you



Rights to copy

© Copyright 2004-2026, Bootlin
License: Creative Commons Attribution - Share Alike 3.0
https://creativecommons.org/licenses/by-sa/3.0/legalcode
You are free:

- to copy, distribute, display, and perform the work
- to make derivative works
- to make commercial use of the work

Under the following conditions:

- **Attribution**. You must give the original author credit.
- **Share Alike**. If you alter, transform, or build upon this work, you may distribute the resulting work only under a license identical to this one.
- For any reuse or distribution, you must make clear to others the license terms of this work.
- Any of these conditions can be waived if you get permission from the copyright holder.

Your fair use and other rights are in no way affected by the above.

Document sources: https://github.com/bootlin/training-materials/

Backup slides



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



mmap



mmap

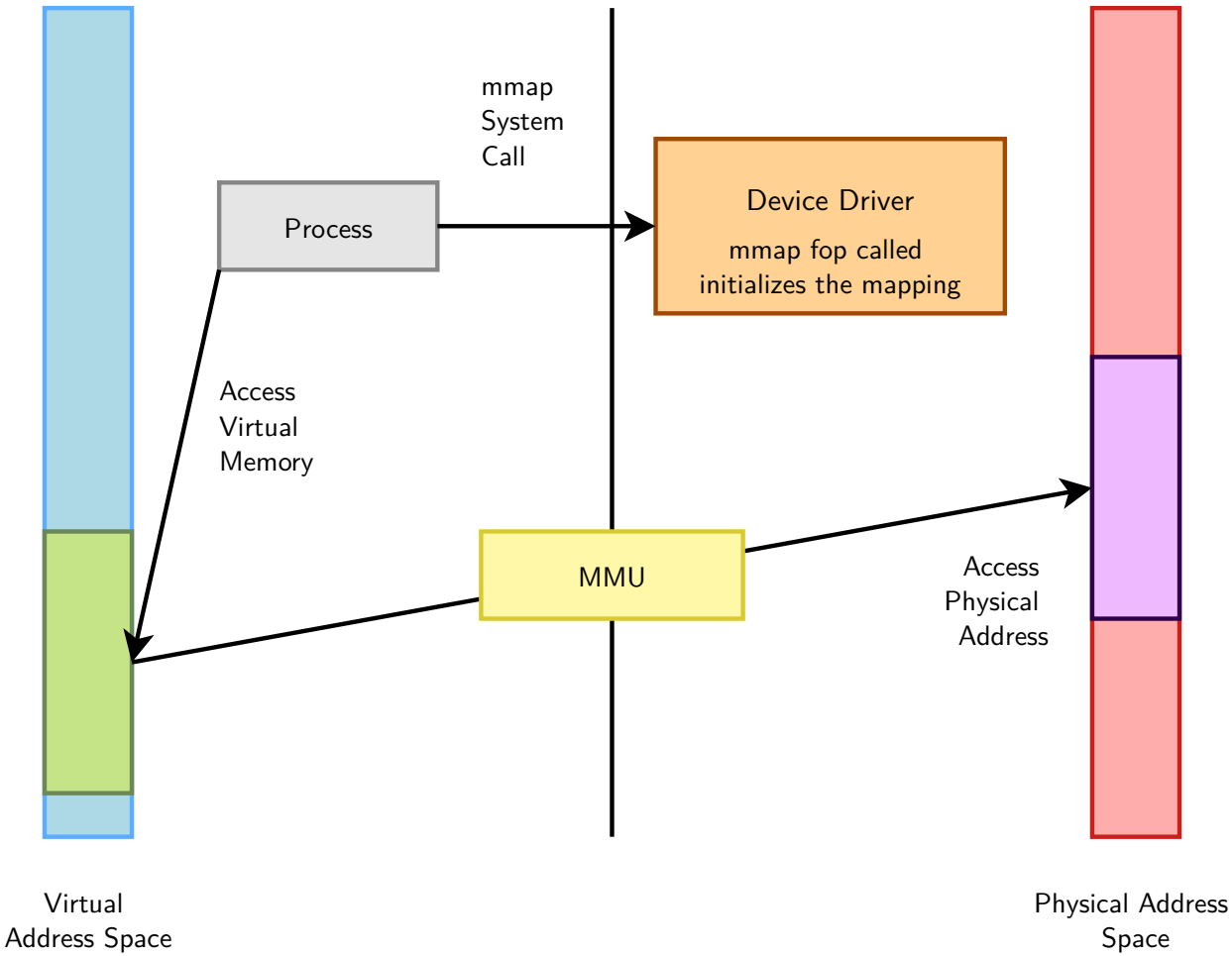
- Possibility to have parts of the virtual address space of a program mapped to the contents of a file
- Particularly useful when the file is a device file
- Allows to access device I/O memory and ports without having to go through (expensive) read, write or ioctl calls
- One can access to current mapped files by two means:
  - `/proc/<pid>/maps`
  - `pmap <pid>`



/proc/<pid>/maps

```
       start-end          perm offset major:minor inode  mapped file name
...
7f4516d04000-7f4516d06000 rw-s 1152a2000 00:05 8406   /dev/dri/card0
7f4516d07000-7f4516d0b000 rw-s 120f9e000 00:05 8406   /dev/dri/card0
...
7f4518728000-7f451874f000 r-xp 00000000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
7f451874f000-7f451894f000 ---p 00027000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
7f451894f000-7f4518951000 r--p 00027000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
7f4518951000-7f4518952000 rw-p 00029000 08:01 268909  /lib/x86_64-linux-gnu/libexpat.so.1.5.2
...
7f451da4f000-7f451dc3f000 r-xp 00000000 08:01 1549    /usr/bin/Xorg
7f451de3e000-7f451de41000 r--p 001ef000 08:01 1549    /usr/bin/Xorg
7f451de41000-7f451de4c000 rw-p 001f2000 08:01 1549    /usr/bin/Xorg
...
```

mmap overview





How to Implement mmap - User space

- Open the device file
- Call the `mmap` system call (see `man mmap` for details):

```
void * mmap(
    void *start,   /* Often 0, preferred starting address */
    size_t length, /* Length of the mapped area */
    int prot,      /* Permissions: read, write, execute */
    int flags,     /* Options: shared mapping, private copy... */
    int fd,        /* Open file descriptor */
    off_t offset   /* Offset in the file */
);
```

- You get a virtual address you can write to or read from.



How to Implement mmap - Kernel space

- Character driver: implement an `mmap` file operation and add it to the driver file operations:

```
int (*mmap) (
    struct file *,           /* Open file structure */
    struct vm_area_struct *  /* Kernel VMA structure */
);
```

- Initialize the mapping.
  - Can be done in most cases with the remap_pfn_range() function, which takes care of most of the job.



remap_pfn_range()

- *pfn*: page frame number
- The most significant bits of the page address (without the bits corresponding to the page size).

```
#include <linux/mm.h>

int remap_pfn_range(
    struct vm_area_struct *, /* VMA struct */
    unsigned long virt_addr, /* Starting user
                              * virtual address */
    unsigned long pfn,       /* pfn of the starting
                              * physical address */
    unsigned long size,      /* Mapping size */
    pgprot_t prot            /* Page permissions */
);
```



Simple mmap implementation

```
static int acme_mmap
    (struct file * file, struct vm_area_struct *vma)
{
      size = vma->vm_end - vma->vm_start;

      if (size > ACME_SIZE)
          return -EINVAL;

      if (remap_pfn_range(vma,
                  vma->vm_start,
                  ACME_PHYS >> PAGE_SHIFT,
                  size,
                  vma->vm_page_prot))
          return -EAGAIN;

      return 0;
    }
```



devmem2

- https://bootlin.com/pub/mirror/devmem2.c, by Jan-Derk Bakker
- Very useful tool to directly peek (read) or poke (write) I/O addresses mapped in physical address space from a shell command line!
  - Very useful for early interaction experiments with a device, without having to code and compile a driver.
  - Uses `mmap` to `/dev/mem`.
  - Examples (`b`: byte, `h`: half, `w`: word)
    - `devmem2 0x000c0004 h` (reading)
    - `devmem2 0x000c0008 w 0xffffffff` (writing)
  - `devmem` is now available in BusyBox, making it even easier to use.



mmap summary

- The device driver is loaded. It defines an `mmap` file operation.
- A user space process calls the `mmap` system call.
- The `mmap` file operation is called.
- It initializes the mapping using the device physical address.
- The process gets a starting address to read from and write to (depending on permissions).
- The MMU automatically takes care of converting the process virtual addresses into physical ones.
- Direct access to the hardware without any expensive `read` or `write` system calls



Useful general-purpose kernel APIs



Memory/string utilities

- In include/linux/string.h
  - Memory-related: memset(), memcpy(), memmove(), memscan(), memcmp(), memchr()
  - String-related: strcpy(), strcat(), strcmp(), strchr(), strrchr(), strlen() and variants
  - Allocate and copy a string: kstrdup(), kstrndup()
  - Allocate and copy a memory area: kmemdup()
- In include/linux/kernel.h
  - String to int conversion: simple_strtoul(), simple_strtol(), simple_strtoull(), simple_strtoll()
  - Other string functions: sprintf(), sscanf()



Linked lists

- Convenient linked-list facility in include/linux/list.h
  - Used in thousands of places in the kernel
- Add a struct list_head member to the structure whose instances will be part of the linked list. It is usually named `node` when each instance needs to only be part of a single list.
- Define the list with the LIST_HEAD() macro for a global list, or define a struct list_head element and initialize it with INIT_LIST_HEAD() for lists embedded in a structure.
- Then use the `list_*()` API to manipulate the list
  - Add elements: list_add(), list_add_tail()
  - Remove, move or replace elements: list_del(), list_move(), list_move_tail(), list_replace()
  - Test the list: list_empty()
  - Iterate over the list: `list_for_each_*()` family of macros



Linked lists examples 1/2

drivers/i2c/busses/i2c-stm32f7.c

```
/**
 * struct stm32f7_i2c_timings - private I2C output parameters
 * @node: List entry
 * @presc: Prescaler value
 * @scldel: Data setup time
 * @sdadel: Data hold time
 * @sclh: SCL high period (master mode)
 * @scll: SCL low period (master mode)
 */
struct stm32f7_i2c_timings {
  struct list_head node;
  u8 presc;
  u8 scldel;
  u8 sdadel;
  u8 sclh;
  u8 scll;
};
```



Linked lists examples 2/2

drivers/i2c/busses/i2c-stm32f7.c

```
static int stm32f7_i2c_compute_timing(/* ... */)
{
  struct stm32f7_i2c_timings *v;
  struct list_head solutions;
  INIT_LIST_HEAD(&solutions);
  /* ... */

  for (p = 0; p < STM32F7_PRESC_MAX; p++) {
    for (l = 0; l < STM32F7_SCLDEL_MAX; l++) {
      v = kmalloc(sizeof(*v), GFP_KERNEL);
      v->presc = p;
      v->scldel = l;
      list_add_tail(&v->node, &solutions);
    }
  }

  list_for_each_entry(v, &solutions, node) {
    /* ... */
  }
}
```